SIMATS ENGINEERING

Assignment

COURSE NAME	Data Structures	COURSE CODE	CSA0304

Register Number	192572083A
Name	NISHA S

Assignment questions

Part	Question
a)	What is tail recursion?
b)	Design algorithm for merging two arrays efficiently.
c)	Explain importance of data structures in cybersecurity.

a) What is tail recursion?

Definition and key idea

Tail recursion is a form of recursion in which a function directly returns the result of its recursive call. No computation remains after that call returns. The recursive call is therefore in the **tail position**. A base case stops the recursion, and an accumulator can carry the partial result into the next call. [1]

For example, ordinary factorial uses `return n * FACT(n - 1)`. The multiplication must wait for the recursive result, so this is **not** tail recursive. Moving the multiplication into an accumulator makes the recursive call the final operation.

Pseudocode: factorial using an accumulator

```
FACTORIAL(n):
    require n is an integer and n >= 0
    return FACT_TAIL(n, 1)

FACT_TAIL(n, acc):
    if n == 0:
        return acc
    return FACT_TAIL(n - 1, acc * n)
```

The product `acc * n` is computed **before** the next call. For an original input N, the invariant **acc x n! = N!** holds at every step. When n becomes zero, acc contains N!, since 0! = 1.

Worked example: FACTORIAL(4)

Call	n	acc	Next action
FACT_TAIL(4, 1)	4	1	Pass (3, 4)
FACT_TAIL(3, 4)	3	4	Pass (2, 12)
FACT_TAIL(2, 12)	2	12	Pass (1, 24)
FACT_TAIL(1, 24)	1	24	Pass (0, 24)
FACT_TAIL(0, 24)	0	24	Return 24

Complexity and significance

There are n recursive steps, so time is **O(n)** under the usual constant-time arithmetic model. Without tail-call optimization, the calls still need **O(n) stack space**. With tail-call optimization, the runtime can reuse a stack frame, giving **O(1) auxiliary stack space**. This optimization depends on the language and implementation; tail recursion alone does not guarantee constant memory.

Tail recursion is useful for processing lists and maintaining running totals. It can be translated into a loop that updates n and acc, which also uses O(1) auxiliary space. For factorial, fixed-width integers may overflow; arbitrary-precision arithmetic has costs that grow with the number of digits.

b) Merge two arrays efficiently

Problem specification and assumption

Given arrays A and B of lengths m and n, produce C containing all m + n elements in nondecreasing order. The main algorithm assumes that **both input arrays are already sorted**. It preserves duplicates and leaves the input arrays unchanged. If the arrays are unsorted, sorting or simple concatenation is required depending on the desired output, as explained on the next page.

Method: two pointers

Maintain indices i and j for the first unprocessed elements of A and B, and k for the next output position. Compare A[i] with B[j], copy the smaller into C, and advance the corresponding input index and k. When one input ends, copy all remaining elements of the other. Each input element is copied once. [2]

Pseudocode: zero-based indexing

```
MERGE_SORTED(A, B):
    m = length(A)
    n = length(B)
    C = new array of length m + n
    i = 0; j = 0; k = 0

    while i < m and j < n:
        if A[i] <= B[j]:
            C[k] = A[i]
            i = i + 1
        else:
            C[k] = B[j]
            j = j + 1
        k = k + 1

    while i < m:
        C[k] = A[i]
        i = i + 1; k = k + 1

    while j < n:
        C[k] = B[j]
        j = j + 1; k = k + 1

    return C
```

Why the algorithm is correct

Loop invariant: before each comparison, k = i + j, and C[0..k-1] is the sorted merge of the consumed prefixes A[0..i-1] and B[0..j-1]. Every element written is no greater than any element still unprocessed. Initially, these prefixes are empty, so the invariant holds.

Since each input is sorted, its first unprocessed element is its smallest remaining element. Selecting the smaller head therefore preserves the invariant. Every iteration advances an index, so the loop terminates. The remaining suffix is already sorted and can be appended. Thus C contains every input element exactly once in sorted order.

b) Worked example and complexity (continued)

Input: A = [1, 4, 7, 9] and B = [2, 4, 6]. The two values equal to 4 are kept; the value from A is selected first on a tie.

Step	Current heads	Action	Output so far
1	1 and 2	Take 1 from A	[1]
2	4 and 2	Take 2 from B	[1, 2]
3	4 and 4	Take 4 from A	[1, 2, 4]
4	7 and 4	Take 4 from B	[1, 2, 4, 4]
5	7 and 6	Take 6 from B	[1, 2, 4, 4, 6]
6	B is exhausted	Copy remaining 7	[1, 2, 4, 4, 6, 7]
7	B is exhausted	Copy remaining 9	[1, 2, 4, 4, 6, 7, 9]

Time and space analysis

Measure	Result and reason
Time: best, average, worst	O(m + n), and tightly Theta(m + n) for nonempty total input: every element is written exactly once.
Element comparisons	At most m + n - 1 when both arrays are nonempty; remaining elements need no cross-array comparisons.
Storage	O(m + n) for the output array. O(1) auxiliary working space if the required output array is excluded.
Stability	The <= test chooses A first on equality. Relative order within each input is retained; this is stable when A precedes B in the original ordering.

The running time is optimal when a separate merged array must be materialized: writing m + n output elements already requires Omega(m + n) work. Simply concatenating and sorting would take O((m + n) log(m + n)) with a comparison sort, wasting the existing sorted order.

Boundary cases and other requirements

Empty arrays: if one array is empty, copy the other; if both are empty, return an empty array. Unequal lengths and negative numbers need no special handling. Equality must preserve both copies unless duplicate removal is explicitly required.

Unsorted inputs: if a sorted result is required, sort A and B first and then merge. Using comparison sorts gives O(m log m + n log n + m + n) time overall. If ordering is unnecessary, concatenate the arrays in O(m + n) time instead.

Space-saving variant: if A has capacity for m + n elements, merge backward using i = m - 1, j = n - 1, and k = m + n - 1. Write the larger remaining value into A[k]. This protects unread values and uses O(m + n) time and O(1) extra space.

c) Data structures in cybersecurity

Why data structures matter

Cybersecurity systems process network packets, login attempts, access permissions, file indicators, and event logs. Data structures organize this information so that security tools can search, update, correlate, and prioritize it quickly. A good choice reduces detection delay and memory use while supporting correct decisions under heavy traffic.

Important structures and practical applications

Data structure	Cybersecurity application	Typical cost / qualification
Hash table / hash set	Store blocked IP addresses, known malicious file digests, session records, or failure counters. Fast exact lookup helps check each event.	Expected O(1) lookup and update with suitable hashing and load control; worst case O(N) for simple collision handling. [3]
Balanced search tree	Index security logs by time and retrieve events within a suspicious interval. Ordered keys support range queries.	O(log N) search / insert / delete; range retrieval O(log N + r) for r returned records.
Trie / prefix tree	Match IP address prefixes or stored string prefixes against filtering rules. Shared prefixes avoid scanning every rule.	O(L) lookup for a key of length L, assuming constant-time child access. Tries organize keys by prefixes. [4]
Queue and stack	A queue buffers security events in arrival order. A stack supports nested-message parsing and depth-first investigation of relationships.	O(1) enqueue / dequeue or push / pop with suitable implementations; capacity limits prevent unbounded growth.
Graph	Represent devices, accounts, and connections to investigate suspicious communication or possible attack paths.	Adjacency lists use O(V + E) space. Breadth-first or depth-first traversal takes O(V + E).
Priority queue / heap	Process high-severity alerts first, or retrieve the next session-expiry time efficiently.	A binary heap supports O(log N) insert / removal and O(1) access to the highest-priority item.

Notation: N is the number of stored records, L is key length, V is the number of graph vertices, and E is the number of edges. The table gives illustrative applications; the same security system may combine several structures.

Benefits for security operations

Speed: indexed lookups avoid scanning large lists for every event. **Scalability:** queues absorb short traffic bursts and efficient storage handles more records. **Investigation:** ordered indexes and graphs connect related evidence. **Availability:** bounded structures and predictable operation costs help prevent resource exhaustion.

c) Example and implementation considerations (continued)

Example: detecting repeated failed logins

Use a hash map keyed by account and time window. Each value counts failed logins. In this simplified example, an alert is created on the fifth failure within a fixed 60-second window. The numbers illustrate the method; they are not a universal security policy.

```
ON_FAILED_LOGIN(account, time_seconds):
    window = floor(time_seconds / 60)
    key = (account, window)
    failures[key] = failures.get(key, 0) + 1
    if failures[key] == 5:
        enqueue(alerts, (account, window, "review"))

PERIODIC_CLEANUP(current_window):
    remove entries from expired windows
```

Five failures for one account at seconds 10, 15, 22, 30, and 41 have the same window number. The counter becomes 1, 2, 3, 4, and 5, producing one alert. A hash map provides expected **O(1) work per event**, and a linked queue provides O(1) alert insertion. Counter storage is **O(U)** for U retained account-window pairs; the alert queue uses O(Q) for Q queued alerts. A full cleanup scan costs O(U).

A fixed window can split closely spaced attempts across a boundary. A sliding-window approach can address this limitation. A practical system also needs atomic counter updates, expiry, bounded queues, and multiple signals. Distributed attempts may evade an IP-only detector, and automatic account lockout can itself deny access to legitimate users. [5]

Choosing and implementing structures safely

Data structures support security but do not provide security by themselves. Check array bounds, validate inputs, synchronize shared updates, and protect log integrity and access. Defend hash tables against excessive collisions and limit stored entries. A hash-table indexing function is **not** a substitute for cryptographic hashing or password protection. Choose structures according to required operations, worst-case behavior, and available memory.

Together, hash maps, trees, queues, stacks, graphs, and heaps enable faster monitoring and better investigation. Their security value comes from combining efficient organization with correct access controls, validation, and resource management.

References

[1] Cornell University. *CS 312 Lecture 5: Folding and tail recursion.*
https://www.cs.cornell.edu/courses/cs312/2008sp/recitations/rec05.html

[2] Sedgewick, R., and Wayne, K. *Algorithms, 4th Edition, Section 2.2: Mergesort.* Princeton University.
https://algs4.cs.princeton.edu/22mergesort/index.php

[3] Black, P. E. *Hash table.* NIST Dictionary of Algorithms and Data Structures.
https://xlinux.nist.gov/dads/HTML/hashtab.html

[4] Black, P. E. *Trie.* NIST Dictionary of Algorithms and Data Structures.
https://xlinux.nist.gov/dads/HTML/trie.html

[5] OWASP Foundation. *Blocking Brute Force Attacks.*
https://community.owasp.org/controls/Blocking_Brute_Force_Attacks